

Lab 5 Value-Based RL

314551022 廖洺玄

Introduction

這個 Lab 主要就是要學怎麼用 DQN 去做 RL 並跑在遊戲 Pong, CartPole 上，Task 1 和 2 差別不大就是 implement Vanilla DQN，然後針對 DQN 的 NN 針對 input layer 做調整。但是 Task 3 就比較麻煩，我 implement 了 Rainbow DQN（6 個 component: Double DQN、PER、Dueling DQN、Multi-step Learning、Distributional RL (C51)、Noisy Nets）然後發現他可以比 Task 2 train 速度快 20 倍以上，我 Task 2 要用 5000 萬步才能收斂到 19 分的模型，我用 Task 3 70~80 萬步就可以了。在這個 Lab 我有用到 Gemini 的引導是學習去一步步叫我 DQN 和 Rainbow DQN 的每個 component，他也幫我改個一些 code 的 bug。

!!!!Evaluation 有在 code 資料夾提供三題的 shell script 請助教在 LAB5_314551022_Code 資料夾中執行三個 shell script 如果要換參數要去 shell script 裡換，或是改 test_model_task{number}.py 的 default value。!!!!

Your implementation

Task 1: Vanilla DQN

我在 DQN model 這裡用了 Fully Connected Network，在裡面我設計了兩個 hidden layer，首先先用他給個 input dimension (他的 states) 做兩次 128 neuron 的 convolutions 後輸出成對訂數量的行動（base on num_actions）。

```
##### YOUR CODE HERE (5~10 lines) #####
self.network = nn.Sequential(
    nn.Linear(input_dim, 128),
    nn.ReLU(),
    nn.Linear(128, 128),
    nn.ReLU(),
    nn.Linear(128, num_actions)
)
##### END OF YOUR CODE #####
```

在這裡我們 init buffer，然後用 DQN 做出來的 output 去 query 好的 action 後存到 buffer 中讓 train 到後面 model 可以 recall 前面的知識。

```

##### YOUR CODE HERE (<5 lines) #####
# Sample a mini-batch of (s,a,r,s',done) from the replay buffer
transitions = random.sample(self.memory, self.batch_size)
states, actions, rewards, next_states, dones = zip(*transitions)

##### END OF YOUR CODE #####

def select_action(self, state):
    if random.random() < self.epsilon:
        return random.randint(0, self.num_actions - 1)
    state_tensor = torch.from_numpy(np.array(state)).float().unsqueeze(0).to(self.device)
    with torch.no_grad():
        q_values = self.q_net(state_tensor)
    return q_values.argmax().item()

```

做完這些前置之後我們會開始 forward pass，我們首先用預測網路（q_net）計算到下一個 state 的 q_value，然後去用選最好的 action。接下來就 apply bellman equation，

```

q_values = self.q_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)

##### YOUR CODE HERE (~10 lines) #####
# Implement the loss function of DQN and the gradient updates
with torch.no_grad():
    next_q_values = self.target_net(next_states).max(1)[0]
    target_q_values = rewards + (1 - dones) * self.gamma * next_q_values

loss = nn.MSELoss()(q_values, target_q_values)

self.optimizer.zero_grad()
loss.backward()
self.optimizer.step()
|

```

How do you obtain the Bellman error for DQN?

Bellman Equation

In my code corresponding to R: rewards, gamma = self.gamma, maxQ(s',a')=next_q_values

$$Q_{new}(s, a) \leftarrow Q_{old}(s, a) + \alpha \cdot [R + \gamma \cdot \max_{a'} Q(s', a') - Q_{old}(s, a)]$$

How do you modify DQN to Double DQN?

一開始要把 DQN 把它改成 Double DQN 就是把原本選 greedy 選 action 的事情交給穩定客觀的 target network 衡量後再選好的 q value 去移動，我這邊 implement 只用簡單的兩行就可以搞定主要就是讓 next_actions 和 next_q_values 那兩行

```

with torch.no_grad():
    # 因為我們存的是 (s_t, a_t, R_t^{(n)}, s_{t+n}, d_{t+n})
    # 所以這裡的 next_states 就是 s_{t+n}
    next_actions = self.q_net(next_states).argmax(1)
    next_q_values = self.target_net(next_states).gather(1, next_actions.unsqueeze(1)).squeeze(1)

    # 如果 n 步後遊戲就結束了，那未來價值就是 0
    next_q_values[dones == 1.0] = 0.0

    # Target = R_t^{(n)} + gamma^n * Q(s_{t+n}, a')
    # rewards 已經是 n-step reward (R_t^{(n)})
    target_q_values = rewards + (self.gamma ** self.n_step) * next_q_values

```

How do you implement the memory buffer for PER?

PER 最主要的功能就是用 priority 有效率地去改善之前錯誤大的地方，也就是透過優先抽樣給那些高 TD-Error 的 exp 做標記讓它們更容易被抽中，最主要是 sample 和 update_priorities 這個 function，然後我用 self.buffer = [None] * capacity list 來存儲 exp。用 TD error 大小決定經驗被採樣的機率 (probs = priorities ** self.alpha)，然後用重要性採樣權重 (weights = (total * probs[indices]) ** (-self.beta)) 在計算損失時校正這種偏差，讓模型既能多學習重要經驗又不會產生偏差。

```

return
def sample(self, batch_size):
    ##### YOUR CODE HERE (for Task 3) #####
    if self.size == 0:
        return [], [], []

    # 根據優先級計算抽樣機率
    priorities = self.priorities[:self.size]
    probs = priorities ** self.alpha
    probs /= probs.sum()

    # 根據機率進行加權抽樣，得到索引
    indices = np.random.choice(self.size, batch_size, p=probs)

    # 計算重要性採樣權重 (IS weights)
    total = self.size
    weights = (total * probs[indices]) ** (-self.beta)
    weights /= weights.max() # 標準化權重

    # 根據索引取得經驗
    batch = [self.buffer[i] for i in indices]

    return batch, indices, torch.tensor(weights, dtype=torch.float32)
    ##### END OF YOUR CODE (for Task 3) #####
    return
def update_priorities(self, indices, errors):
    ##### YOUR CODE HERE (for Task 3) #####
    # 根據新的 TD-Error 更新經驗的優先級
    # 加上一個極小值避免優先級為 0
    for i, error in zip(indices, errors):
        self.priorities[i] = error + 1e-5
    ##### END OF YOUR CODE (for Task 3) #####
def __len__(self):
    return self.size

```

How do you modify the 1-step return to multi-step return?

我修改了 1-step return 為 3-step return：用 deque 收集連續 3 步的經驗，計算折扣累積獎勵 $R(t) + \gamma R(t+1) + \gamma^2 R(t+2)$ ，並將目標狀態從下一步改為第 3 步後的狀態，讓每次更新都能利用更多未來信息進行學習。最關鍵的部分就是下圖去計算多步的獎勵然後 apply 好的。還有下面第二張圖也是很重要的一部分就是每走一步都放到 buffer 裡然後當 buffer 滿了開始選好的後清空。註解掉的地方就是之前 vanilla DQN 的時候，他用隨機的方式選一步來 recall 這樣學習效率就很不好。

```
if done:
    while (variable) n: int iffer) > 0:
        n = len(self.n_step_buffer)
        n_step_reward = sum([self.gamma**i * self.n_step_buffer[i][2] for i in range(n)])
        oldest_state, oldest_action, _, _, _ = self.n_step_buffer[0]
        _, _, _, final_next_state, final_done = self.n_step_buffer[-1]

        self.memory.add(oldest_state, oldest_action, n_step_reward, final_next_state, final_done)
        # 移除最舊的，直到清空
        self.n_step_buffer.popleft()
# self.memory.append((state, action, reward, next_state, done)) without prioritization
# self.memory.add(state, action, reward, next_state, done)
self.n_step_buffer.append((state, action, reward, next_state, done))
if len(self.n_step_buffer) == self.n_step:
    n_step_reward = sum([self.gamma**i * self.n_step_buffer[i][2] for i in range(self.n_step)])
    oldest_state, oldest_action, _, _, _ = self.n_step_buffer[0]
    _, _, _, final_next_state, final_done = self.n_step_buffer[-1]
    self.memory.add(oldest_state, oldest_action, n_step_reward, final_next_state, final_done)
```

Explain how you use Weight & Bias to track the model performance.

我使用 wandb 追蹤四個東西：訓練進度（環境步數、更新次數、epsilon 衰減）、每個 episode 的總獎勵、訓練損失的統計分布（平均值、最大值、最小值、標準差），以及定期評估獎勵，讓我能看 DQN 模型的學習效果和收斂情況，其實就是助教給的 code 再加個幾行而已。

```
print(f'[Eval] Ep: {ep} Total Reward: {total_reward} SC: {self.env_count} UC: {self.train_count} Eps: {self.epsilon:.4f}')
wandb.log({
    "Episode": ep,
    "Total Reward": total_reward,
    "Env Step Count": self.env_count,
    "Update Count": self.train_count,
    "Epsilon": self.epsilon
})
##### YOUR CODE HERE #####
# Add additional wandb logs for debugging if needed

wandb.log({
    "Episode": ep,
    "Total Reward": total_reward,
    "Env Step Count": self.env_count,
    "Update Count": self.train_count,
    "Epsilon": self.epsilon
})

if self.episode_losses: # 確保不是空 list 才計算 loss 統計
    avg_loss = sum(self.episode_losses) / len(self.episode_losses)
    max_loss = max(self.episode_losses)
    min_loss = min(self.episode_losses)
    std_loss = np.std(self.episode_losses)

    wandb.log({
        "Episode": ep,
        "Avg Loss": avg_loss,
        "Max Loss": max_loss,
        "Min Loss": min_loss,
        "Std Loss": std_loss
    })

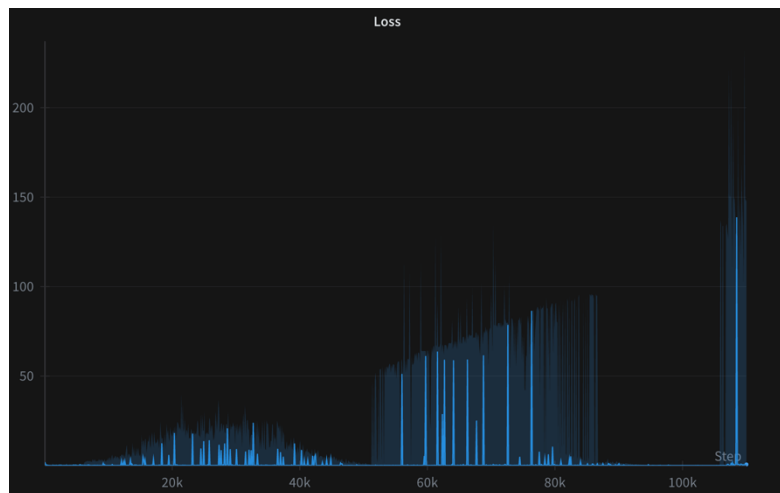
self.episode_losses.clear()
```

Analysis and discussions

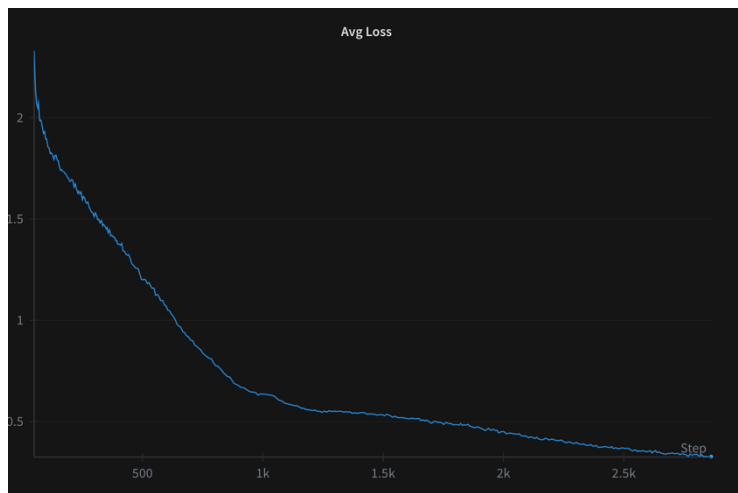
Training curves

從這三個 loss curve 可以看到，task 1 的在 90k 的時候 loss 最少、60~70 的時候最多，task 2 的則是一開始慢慢變多，到整體訓練流程的 1/3 時候開始收斂。而 Task 3 則最理想，從大到小慢慢開始收斂。

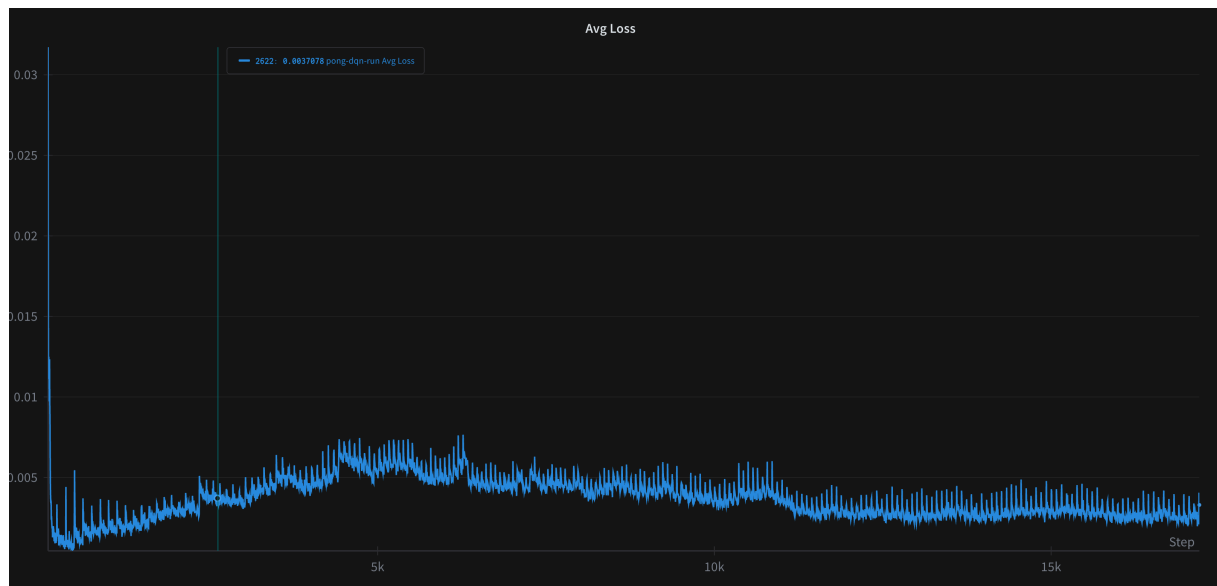
Task 1



Task 3



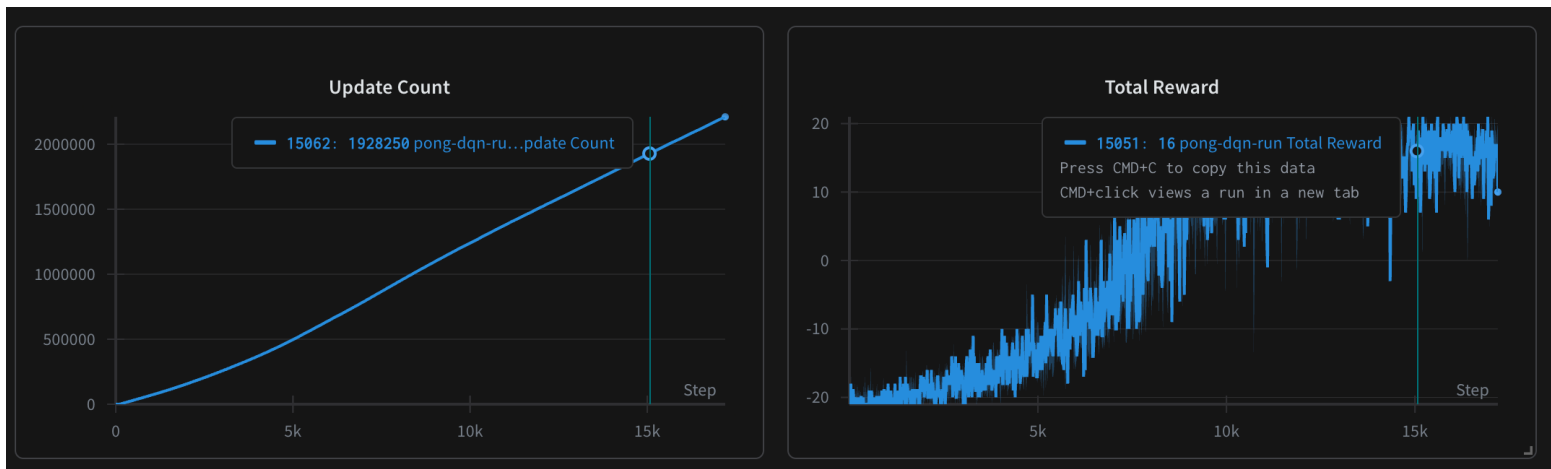
Task 2



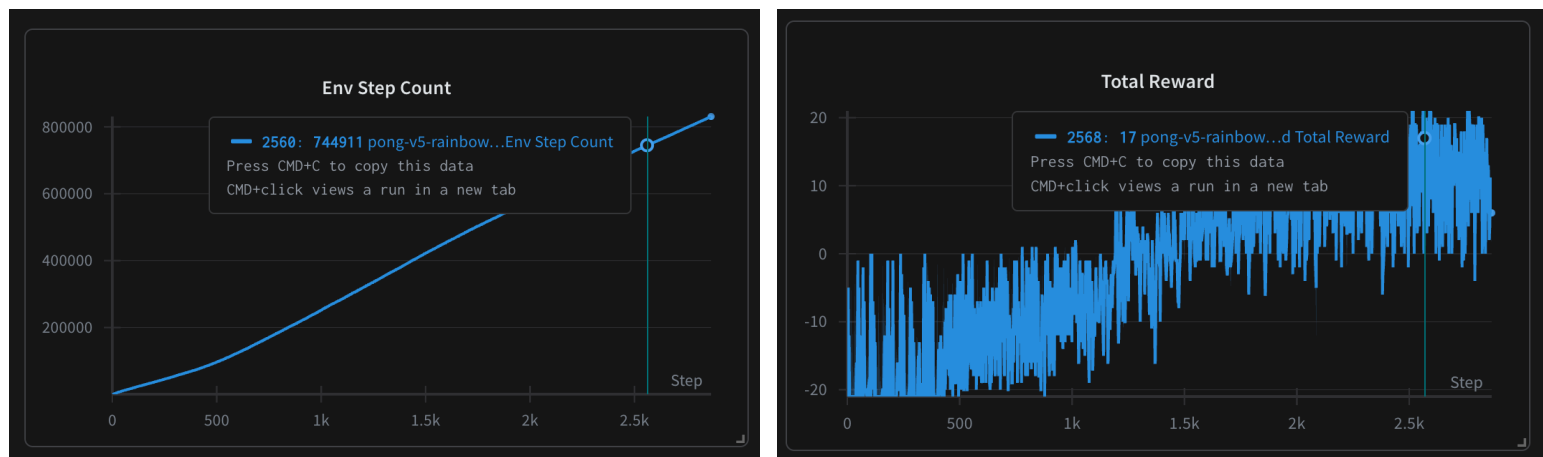
The sample efficiency with and without the DQN enhancements

從這兩個可以看到在我們用了 6 個 component 的 Rainbow DQN 取代 Vanilla DQN 後我們訓練到收斂 avg 19 score 的步數從 2000 萬步下降到了 74 萬步提升了 27 倍。

Task 2



Task3



Result

Task 1

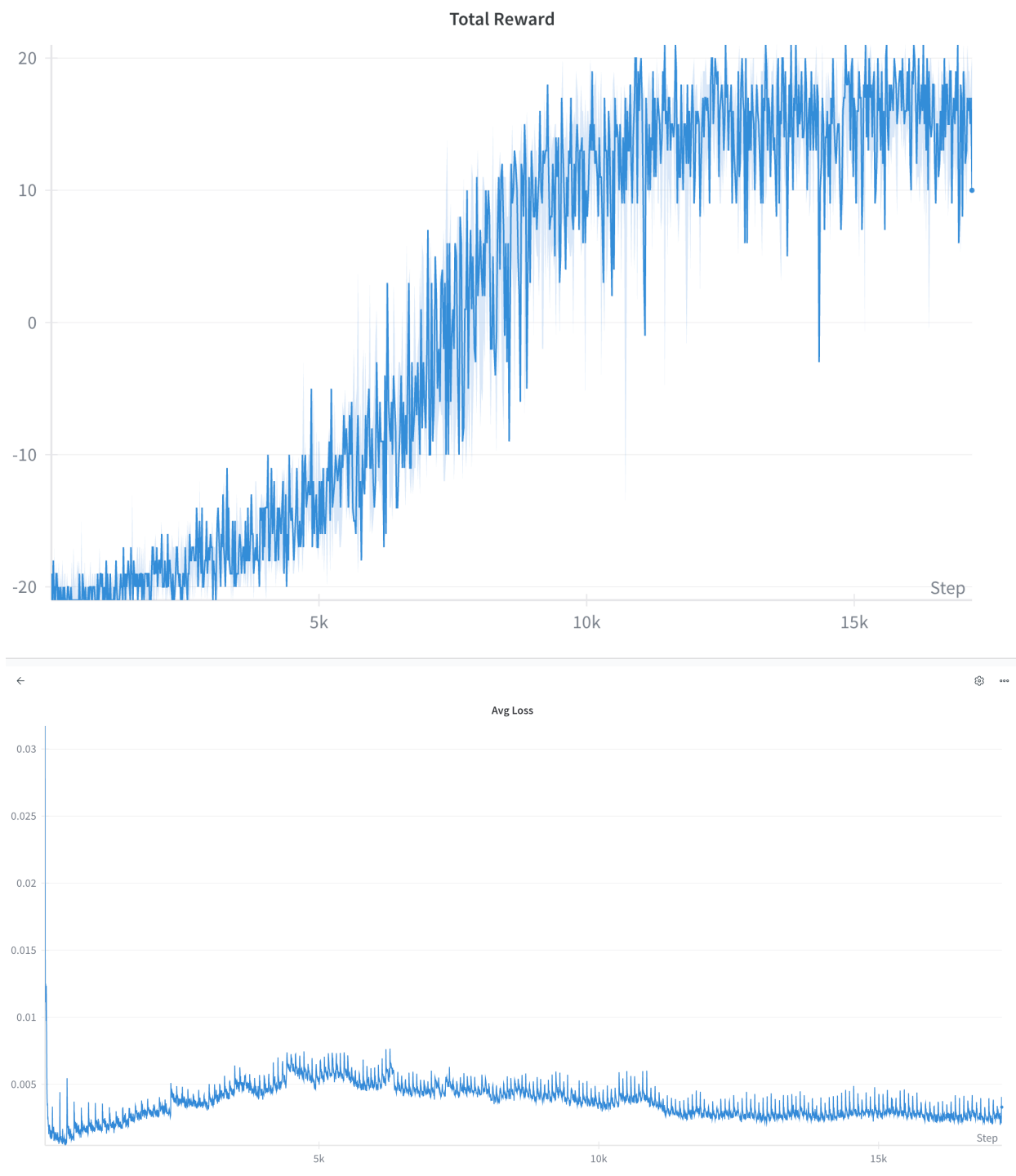
Average 5000, All seed

[illegible]

Task2

Average 19.3, Seed 8382~8401

```
=== Evaluation Summary ===
Total seeds evaluated: 20
Seeds used: [8382, 8383, 8384, 8385, 8386, 8387, 8388, 8389, 8390, 8391, 8392, 8393, 8394, 8395, 8
396, 8397, 8398, 8399, 8400, 8401]
Average reward: 19.30
Std reward: 1.76
Min reward: 15.00
Max reward: 21.00
Rewards: [16.0, 19.0, 21.0, 17.0, 21.0, 20.0, 18.0, 21.0, 15.0, 18.0, 18.0, 20.0, 19.0, 21.0, 21.0
, 20.0, 19.0, 20.0, 21.0, 21.0]
```

Task 3

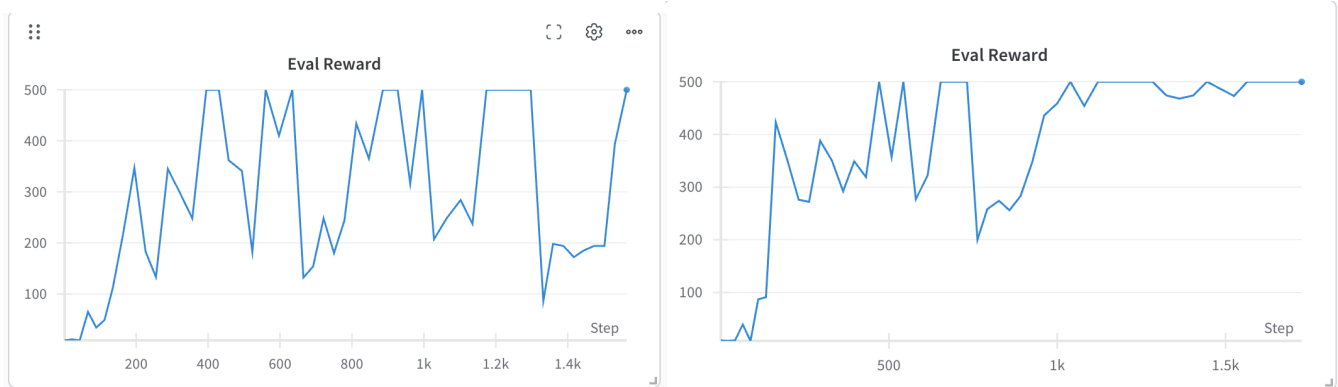
我測試到最好的 model 是 360 epoch 的然後 seed 是 186~205，然後 step 是在 763000



An ablation study on each technique separately

Double DQN

我們把原本 greedy 會直接選最好的下一步的這個方法改成讓穩定的 `target_net` 去評估下一步的價值，就變成是給另一個人客觀的去評估這個 `action` 的實際價值，然後再選好的。我把它 implement 到我的 `task 1` 時候發現他在 `train` 的時候比較容易 `train` 到最好的 `model weight`（左邊是 Vanilla NQN，右邊是 Double DQN 的 `train evaluation` 過程，可以看到 Double DQN 穩定和表現都好很多）



Additional analysis on other training strategies

一. 在 `Task1` 的時候，用兩層 `hidden layer` 是因為我有測過用 1 和 3 層的效果都沒 2 層好，1 層學到的不夠多、3 層會有點 `overfit` 而且 `train` 時間變長。

3 hidden layer leads to sometimes overfit

```
(venv) (base) root@jeff-MS-7042:~/skfsl# python task1_eval20.py --model-path ./results/best_model.pt
Using device: cuda
[Seed 702] Episode 1: Reward = 500.0
[Seed 682] Episode 2: Reward = 500.0
[Seed 193] Episode 3: Reward = 500.0
[Seed 348] Episode 4: Reward = 500.0
[Seed 97] Episode 5: Reward = 500.0
[Seed 518] Episode 6: Reward = 500.0
[Seed 523] Episode 7: Reward = 500.0
[Seed 732] Episode 8: Reward = 500.0
[Seed 661] Episode 9: Reward = 500.0
[Seed 717] Episode 10: Reward = 500.0
[Seed 392] Episode 11: Reward = 500.0
[Seed 946] Episode 12: Reward = 500.0
[Seed 273] Episode 13: Reward = 500.0
[Seed 137] Episode 14: Reward = 500.0
[Seed 380] Episode 15: Reward = 500.0
[Seed 871] Episode 16: Reward = 500.0
[Seed 76] Episode 17: Reward = 349.0
[Seed 631] Episode 18: Reward = 399.0
[Seed 154] Episode 19: Reward = 500.0
[Seed 409] Episode 20: Reward = 500.0

=== Evaluation Complete ===
Seeds used: [702 682 193 348 97 518 523 732 661 717 392 946 273 137 380 871 76 631 154 409]
Episode rewards: [500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 349.0, 399.0, 500.0, 500.0]
Average Reward: 487.40
Score Percentage: 15.80%
```

Using 2 hidden layer barley overfit while evaluating

```
(venv) (base) root@jeff-MS-7D42:~/skfsl# python task1_eval20.py --model-path ./results/best_model.pt
Using device: cuda
[Seed 205] Episode 1: Reward = 500.0
[Seed 332] Episode 2: Reward = 500.0
[Seed 102] Episode 3: Reward = 500.0
[Seed 897] Episode 4: Reward = 500.0
[Seed 182] Episode 5: Reward = 500.0
[Seed 423] Episode 6: Reward = 500.0
[Seed 427] Episode 7: Reward = 500.0
[Seed 775] Episode 8: Reward = 500.0
[Seed 36] Episode 9: Reward = 500.0
[Seed 121] Episode 10: Reward = 500.0
[Seed 414] Episode 11: Reward = 500.0
[Seed 369] Episode 12: Reward = 500.0
[Seed 304] Episode 13: Reward = 500.0
[Seed 876] Episode 14: Reward = 500.0
[Seed 226] Episode 15: Reward = 500.0
[Seed 87] Episode 16: Reward = 500.0
[Seed 132] Episode 17: Reward = 500.0
[Seed 161] Episode 18: Reward = 500.0
[Seed 373] Episode 19: Reward = 500.0
[Seed 974] Episode 20: Reward = 500.0

=== Evaluation Complete ===
Seeds used: [205 332 102 897 182 423 427 775 36 121 414 369 304 876 226 87 132 161 373 974]
Episode rewards: [500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0]
Average Reward: 500.00
Score Percentage: 15.00%
```

Using 1 hidden layer cannot learn enough feature

```
(venv) (base) root@jeff-MS-7D42:~/skfsl# python task1_eval20.py --model-path ./results/best_model.pt
Using device: cuda
[Seed 593] Episode 1: Reward = 47.0
[Seed 983] Episode 2: Reward = 73.0
[Seed 146] Episode 3: Reward = 74.0
[Seed 14] Episode 4: Reward = 81.0
[Seed 896] Episode 5: Reward = 57.0
[Seed 8] Episode 6: Reward = 62.0
[Seed 396] Episode 7: Reward = 65.0
[Seed 815] Episode 8: Reward = 50.0
[Seed 489] Episode 9: Reward = 60.0
[Seed 278] Episode 10: Reward = 83.0
[Seed 320] Episode 11: Reward = 81.0
[Seed 685] Episode 12: Reward = 57.0
[Seed 764] Episode 13: Reward = 72.0
[Seed 960] Episode 14: Reward = 49.0
[Seed 564] Episode 15: Reward = 71.0
[Seed 853] Episode 16: Reward = 63.0
[Seed 453] Episode 17: Reward = 68.0
[Seed 271] Episode 18: Reward = 62.0
[Seed 290] Episode 19: Reward = 87.0
[Seed 233] Episode 20: Reward = 101.0

=== Evaluation Complete ===
Seeds used: [593 983 146 14 896 8 396 815 489 278 320 685 764 960 564 853 453 271 290 233]
Episode rewards: [47.0, 73.0, 74.0, 81.0, 57.0, 62.0, 65.0, 50.0, 60.0, 83.0, 81.0, 57.0, 72.0, 49.0, 71.0, 63.0, 68.0, 62.0, 87.0, 101.0]
Average Reward: 68.15
Score Percentage: 2.13%
```

Reference

1. https://blog.csdn.net/qq_51320133/article/details/137956365
2. https://www.google.com/search?q=rainbow+DQN&oq=rainbow+DQN&gs_lcrp=EgZjaHJvbWUyBggAEEUYOTIGCAEQRRg8MgYIAhBFGDwyBggDEEUYPDIGCAQQQLhhA0gEINDQ5NWowajGoAgCwAgA&sourceid=chrome&ie=UTF-8#:~:text=Rainbow%3A%20The%20Colorful,%E8%AE%9A%20%C2%B7%201%20%E5%B9%B4%E5%89%8D
3. <https://zhuanlan.zhihu.com/p/261322143>
4. https://blog.csdn.net/qq_44940689/article/details/132224427
5. <https://arxiv.org/abs/1710.02298>